

ABU DHABI UNIVERSITY

College of Engineering

SWE401 - Software Engineering

Course Project - Phase 5

Implementation and Testing

Project Title

SmartStudy - Intelligent Academic Planning System

NO.	STUDENT NAME	STUDENT ID
1.	Ismail Monem	1096000
2.	Ahmed El Jeyid	1093121
3.	Mohamed Ahmed	1091429

Instructor

Dr. Chayanika Sharma

Semester

Summer 2025-2026

Submission Date

3 August 2026

Table of Contents

8. Implementation and Testing	3
8.1 Implementation Overview	3
8.2 Implemented System Architecture	4
8.3 Functional Implementation	5
8.4 Database and Smart Features	7
8.5 Source Control, Build and Run Instructions	8
8.6 Testing and Verification	9
8.7 Known Limitations and Final Validation	10
8.8 Team Contributions	11
References	11

8. Implementation and Testing

8.1 Implementation Overview

SmartStudy was implemented as a Java desktop prototype using JavaFX for the graphical user interface, Java for application logic, MySQL for persistent storage, and JDBC for database access. The implementation translates the requirements and design models from Phases 2-4 into a structured Maven project.

The official Phase 5 deliverable requires an implementation overview, source-control information, and basic build/run instructions. This report documents those elements and the included test assets. The application package contains the source code, FXML layouts, CSS, SQL scripts, automated tests, sample syllabi, and a README file.

Category	Implementation
Programming language	Java 21
User interface	JavaFX with FXML and CSS
Database	MySQL 8+ accessed through JDBC
Build system	Apache Maven
Security	BCrypt password hashing; active-account checks; session invalidation
Document processing	Apache PDFBox for PDF and Apache POI for DOCX
Testing	JUnit tests plus a dependency-free core smoke test
Version control	Git and GitHub; repository URL to be inserted after team upload

Implementation Objectives

- Preserve the layered architecture approved in Phase 4.
- Implement the complete student flow from registration to progress tracking.
- Keep administrator account-management functions separate from academic data.
- Support representative PDF/DOCX syllabus extraction with mandatory student review.
- Generate and regenerate a study plan using grade weight, deadline proximity, estimated effort, and available hours.
- Store all account, course, task, schedule, progress, and notification data in MySQL.

8.2 Implemented System Architecture

The source code follows the five-layer design from Phase 4. JavaFX views communicate with controllers, controllers call service classes, services apply business rules, DAOs perform SQL operations, and MySQL/local file storage persist the results. External LMS and email capabilities remain replaceable adapter boundaries; the prototype uses manual upload and in-app reminders.

Layer	Packages / Files	Responsibility
Presentation	resources/fxml, resources/css	Login, registration, dashboard, courses/tasks, syllabus review, schedule, profile, and administrator screens.
Control	controller	Handles UI events, validation feedback, screen navigation, and user commands.
Application logic	service	Authentication, session control, parsing, scheduling, reminder creation, progress and integration coordination.
Data access	dao	Prepared JDBC queries and transactions for every database table.
Persistence & integration	MySQL, data/syllabi	Relational records, referential integrity, uploaded files, and simulated external-service boundaries.

Project Structure

- src/main/java/com/smartstudy/model - domain entities and enumerations
- src/main/java/com/smartstudy/controller - JavaFX controllers
- src/main/java/com/smartstudy/service - business logic and smart features
- src/main/java/com/smartstudy/dao - JDBC database access
- src/main/resources/fxml - interface layouts
- src/main/resources/css - shared visual style
- database - schema and seed scripts
- sample-syllabi - representative PDF and DOCX files
- src/test - JUnit tests
- tools - source verification and core smoke test

The complete source package is delivered separately as SmartStudy_Phase5.zip.

8.3 Functional Implementation

FR	Implemented behavior	Primary implementation
FR1	Registers a student after required-field, email, duplicate-email, password, and available-hours validation.	RegisterController, AuthService, StudentDao
FR2	Authenticates students and administrators; blocks incorrect credentials and deactivated accounts.	LoginController, AuthService
FR3	Updates full name, password, weekly available hours, and notification preferences.	ProfileController, StudentDao
FR4	Adds, edits, lists, and deletes courses belonging to the signed-in student.	CourseTaskController, CourseDao
FR5	Adds, edits, deletes, filters and completes assignments, exams and quizzes.	CourseTaskController, TaskDao
FR6	Copies PDF/DOCX syllabi into controlled local storage and records the file in MySQL.	SyllabusController, SmartStudyService
FR7	Extracts assessment titles, types, dates and weights; lets the student edit and confirm each item.	SyllabusParser, SyllabusDao
FR8	Creates weekly study sessions using priority and available time.	ScheduleGenerator, ScheduleDao
FR9	Regenerates the remaining plan after task creation, edit, deletion or status change.	SmartStudyService
FR10	Stores Pending, In Progress and Completed status; completed tasks are excluded from future plans.	TaskDao, DashboardController
FR11	Displays course count, weekly study time, progress, upcoming tasks and reminders.	DashboardController
FR12	Creates in-app deadline and study-session reminders when a plan is generated.	NotificationDao, SmartStudyService
FR13	Persists data through MySQL tables with foreign keys, unique constraints and cascading deletion.	database/schema.sql and DAOs
FR14	Clears the active session on logout and checks the inactivity timeout on protected operations.	SessionManager, MainController
FR15	Allows administrators to view, activate, deactivate and delete student accounts.	AdminController, StudentDao

Main User Flow

- The student creates an account and logs in.
- The student adds courses and tasks or uploads a representative syllabus.
- Extracted items are reviewed and corrected before confirmation.
- The system ranks active tasks and generates study blocks for the current week.
- The dashboard displays deadlines, study hours, progress and reminders.
- Changing task information automatically regenerates the plan.
- The student logs out; protected data is inaccessible until a new login.

Administrator Flow

- The administrator signs in using the separate administrator account.
- The application displays only the Administrator Console and Logout actions.
- The administrator can activate, deactivate or permanently delete a selected student account.
- Academic courses, tasks and schedules are not exposed to the administrator interface.

Interface Consistency

All screens share the same blue application header, left navigation panel, typography, table styling and validation feedback. Student and administrator navigation are role-specific so a student cannot open the administrator console.

8.4 Database and Smart Features

Database Implementation

The database script creates nine tables: Student, Administrator, Course, Task, Syllabus, ExtractedItem, Schedule, StudySession, and Notification. The SQL implementation uses the corrected Phase 4 design, including courseId as a surrogate primary key, UNIQUE(studentId, courseCode), Student.isActive, Syllabus.storedPath, and ExtractedItem.title/dueDate.

Constraint	Purpose
UNIQUE students.email	Prevents duplicate accounts.
UNIQUE administrators.email	Prevents duplicate administrator identities.
UNIQUE(student_id, course_code)	Stops one student from adding the same course twice.
UNIQUE syllabi.course_id	Allows at most one stored syllabus per course.
Foreign keys with cascading deletion	Prevents orphaned academic records after course or account deletion.
Task status ENUM	Restricts status to PENDING, IN_PROGRESS or COMPLETED.
Indexes on deadlines/status/reminders	Supports responsive schedule and dashboard queries.

Schedule Generation

For each active task, the scheduler calculates a priority score from grade weight, urgency, and estimated effort. Overdue tasks receive maximum priority. Tasks are sorted from highest to lowest score and divided into study blocks of up to 1.5 hours without exceeding the student's weekly available hours or scheduling a block after its deadline.

$$\text{priority} = 0.50(\text{weight}) + 0.35(\text{urgency}) + 0.15(\text{effort})$$

If there is insufficient time before a deadline, the generator returns a warning rather than silently producing an impossible schedule.

Syllabus Extraction

The parser reads PDF text with PDFBox and DOCX paragraphs with Apache POI. It recognizes representative assessment keywords, common date formats and percentage weights. Because syllabus formats vary, no extracted item is converted into a task until the student reviews and confirms it.

Reminder Creation

Each regeneration replaces future automatic reminders and creates an in-app reminder before every generated study session and active task deadline. The database stores message, time, channel and sent status so a real notification gateway can be connected later.

8.5 Source Control, Build and Run Instructions

Repository

Repository URL: [INSERT TEAM GITHUB REPOSITORY URL]

The team must upload the delivered SmartStudy_Phase5 folder to one GitHub repository. Each member should work on a named feature branch and make visible commits because individual contribution is assessed.

Branch	Purpose
main	Stable demonstration-ready version.
development	Integrated team work before release.
feature/database-auth	Database, authentication and administrator functions.
feature/javafx-ui	FXML, controllers, CSS and navigation.
feature/parser-scheduler	Syllabus extraction, scheduling and tests.

Prerequisites

- JDK 21
- Apache Maven 3.9 or later
- MySQL 8 or later
- Internet access during the first Maven build so required libraries can be downloaded

Database Setup

- Run database/schema.sql in MySQL.
- Run database/seed.sql.
- Open src/main/resources/application.properties.
- Replace CHANGE_ME with the local MySQL password.

Build and Run

```
mvn clean test
mvn javafx:run
```

NetBeans users can open the folder as a Maven project, allow dependencies to download, and run com.smartstudy.SmartStudyApp.

Default administrator for demonstration: admin@smartstudy.local / Admin123!

8.6 Testing and Verification

The package includes JUnit tests, a dependency-free core smoke test, representative PDF/DOCX files, and a source-verification script. The following checks were executed while preparing the package:

Check	Result	Evidence
Core validation rules	PASS	Valid email/password accepted; invalid input throws validation errors.
Priority scheduling	PASS	Higher-weight, nearer-deadline task is scheduled first.
Completed-task exclusion	PASS	Completed tasks produce no future sessions.
Core Java compilation	PASS	Models, DAOs and services compiled with external-library API stubs.
FXML syntax	PASS	All nine FXML files parsed as well-formed XML.
Representative extraction logic	PASS	Rule-based parser test covers Assignment, Exam and Quiz patterns.
Full JavaFX + MySQL execution	TEAM WORKSTATION	Requires Maven dependencies and a configured MySQL server.

Required End-to-End Test Cases

ID	Test	Expected result
TC-01	Register valid student	Account stored and login available.
TC-02	Register duplicate email	Clear rejection; no duplicate row.
TC-03	Student login/logout	Dashboard opens; logout blocks protected screens.
TC-04	Deactivated login	Access rejected.
TC-05	Course CRUD	List updates and persists after restart.
TC-06	Task CRUD/status	Task persists; dashboard and schedule update.
TC-07	Upload valid PDF/DOCX	File stored and items displayed for review.
TC-08	Upload unsupported/corrupt file	Error shown; no invalid record committed.
TC-09	Edit/confirm extracted item	Confirmed item becomes a Task.
TC-10	Generate schedule	Study sessions created in no more than 5 seconds.
TC-11	Automatic regeneration	Task change replaces current plan.
TC-12	Progress update	Completed task immediately changes progress.
TC-13	Reminder generation	Deadline and session reminders are persisted.
TC-14	Persistence/restart	Saved academic data remains available.
TC-15	Administrator operations	View, activate, deactivate and delete work correctly.

Before submission, the team must execute TC-01 through TC-15 on the same computer used for the live demonstration and capture real screenshots. The report must not claim that email or LMS integration was tested unless those services were actually configured.

8.7 Known Limitations and Final Validation

Limitation	Prototype handling
Institutional LMS authentication	Not included. Manual PDF/DOCX upload represents the approved prototype adapter boundary.
Email delivery	Not configured. In-app notification records and delivery status are implemented.
Unstandardized syllabi	Rule-based extraction targets representative documents; student review is mandatory.
Cloud/shared deployment	The approved prototype uses a desktop JavaFX application and local MySQL installation.
Cross-platform verification	Source is portable, but Windows/macOS/Linux execution must be confirmed by the team.
Repository URL	Cannot be generated until the team uploads the folder to its GitHub account.

Final Team Validation Checklist

- ☐ Replace the repository placeholder with the real URL.
- ☐ Configure application.properties.
- ☐ Run schema.sql and seed.sql.
- ☐ Run mvn clean test and mvn javafx:run.
- ☐ Complete TC-01 through TC-15.
- ☐ Capture real screenshots for the final compiled report.
- ☐ Confirm the default administrator account works.
- ☐ Verify that each team member has visible Git commits.

8.8 Team Contributions

Team Member	Implementation responsibility	Evidence to retain
Mohamed Ahmed	Database schema, JDBC DAOs, authentication, password hashing, session handling and administrator account management.	SQL commits, DAO/service commits, database test screenshots.
Ahmed El Jeyid	JavaFX layouts, controllers, navigation, dashboard, courses/tasks, profile and consistent CSS.	FXML/controller commits and interface screenshots.
Ismail Monem	Syllabus upload/extraction, extracted-item review, schedule generator, reminder creation and testing.	Parser/scheduler/test commits and test evidence.

The exact contribution statement should be adjusted to match the team's actual final work and Git history. Do not report work under a member's name unless that member completed or verified it.

References

Apache Maven Project. (n.d.). Maven documentation.

OpenJFX. (n.d.). Getting started with JavaFX.

Oracle. (n.d.). MySQL 8.0 Reference Manual.

Apache Software Foundation. (n.d.). Apache PDFBox and Apache POI documentation.

Provost, D., & Mazières, D. (1999). A future-adaptable password scheme.